Deep Learning and Language Model Pretraining

From neural-network optimization to tokenization, data, scaling, and evaluation

Scope and approach

This document is a structured technical learning guide, written to connect foundational ideas to modern practice. It distinguishes established concepts from fast-moving research directions. It is not a systematic literature review and should be read alongside the original papers and documentation in the references.

How to use this guide

Read sequentially for a course-like progression. For each topic, focus on (1) the problem it solves, (2) the mechanism, (3) trade-offs, and (4) how to test whether it works in a real system.

1. Neural-network foundations

A neural network composes parameterized functions. An embedding maps discrete token IDs to vectors; linear layers transform representations; nonlinearities such as GELU or SwiGLU increase expressiveness; residual connections allow information and gradients to flow through deep stacks. Parameters are learned by minimizing an objective over training examples.

Loss, gradients, and optimization

For next-token prediction, the model produces a probability distribution over the vocabulary. Cross-entropy loss penalizes low probability assigned to the correct next token. Backpropagation computes gradients of the loss with respect to parameters; an optimizer such as Adam or AdamW updates them. Learning rate, batch size, sequence length, data mixture, weight decay, gradient clipping, and warm-up all affect training stability.

Training loss decreasing does not prove useful generalization. Track held-out loss and task evaluations, inspect data leakage, and compare against simple baselines. A model can memorize training examples and still fail to generalize.

2. Tokenization and embeddings

Text must be converted into token IDs. Common subword approaches include byte-pair encoding (BPE), WordPiece, and unigram language-model tokenization. Tokenizers trade vocabulary size against sequence length, rare-word handling, multilingual coverage, and reversibility. Token boundaries are not the same as words; a word can split into multiple tokens.

Embeddings map token IDs to dense vectors. In decoder-only LMs, the output vocabulary projection often shares weights with the input embedding matrix, though this is not universal. Contextual hidden states then depend on the surrounding sequence. Static word vectors such as word2vec were useful historically but cannot represent the same word differently by context.

Practical implications

- Tokenization changes the effective context length and cost for different languages and code.
- A tokenizer mismatch between training and serving is a serious compatibility bug.
- Prompt length should be measured in tokens, not characters or words.
- Embeddings are not automatically a semantic database; retrieval requires an indexing and similarity strategy.

3. Pretraining objectives

Autoregressive next-token prediction

The model estimates P(x■ | x■…x■■■) and minimizes negative log-likelihood across token positions. It naturally supports generation by repeatedly sampling or selecting the next token. It also learns many latent skills from broad text, but the objective is prediction—not a direct guarantee of truth, safety, or grounded reasoning.

Masked-language modeling

A subset of input tokens is hidden or corrupted and the model predicts them using left and right context. BERT is a prominent example. Such models can be excellent encoders for classification and representation tasks, but they are not inherently left-to-right generators in the same way as decoder-only LMs.

Text-to-text and denoising

T5 casts tasks such as translation, summarization, and classification into text input to text output. Denoising objectives reconstruct corrupted spans. Objective choice affects which capabilities are learned naturally and how a model is used downstream.

4. Training data and data governance

Pretraining corpora may combine web text, books, code, scientific writing, and licensed or curated datasets. Quality often matters as much as raw volume. Data pipelines typically deduplicate documents, filter spam and low-quality content, normalize formats, detect language, and remove prohibited or sensitive material where feasible.

Important risks include benchmark contamination, personal information, copyrighted material, toxic content, hidden prompt-injection strings in scraped data, and uneven representation across languages and domains. Dataset documentation should record sources, filtering, known limitations, and intended use.

> Data leakage is not a small technical detail: if evaluation answers or near-duplicates appear in training data, benchmark scores can overstate real-world generalization.

5. Scaling laws and compute trade-offs

Scaling-law research studies how loss changes with model size, data, and compute. Early work emphasized parameter scaling; later compute-optimal work showed that model size and training-token count should be balanced for a fixed compute budget. There is no universal parameter count that makes a model “good”; inference cost, data quality, deployment environment, and target tasks matter.

Budgeting a training run

- Estimate total tokens, sequence length, batch size, steps, optimizer state, activation memory, and checkpoint storage.
- Distinguish training compute from serving cost. A model that is affordable to train may be too expensive to serve at required throughput.
- Measure tokens per second and validation loss early; stop or revise runs that show unstable loss or poor data quality.
- Use small-scale pilot experiments to test data mixtures and learning-rate schedules before expensive runs.

6. Evaluation and generalization

Perplexity is an exponentiated average negative log-likelihood; lower is better on the same tokenization and evaluation distribution, but it is not a complete measure of usefulness. Task benchmarks, human evaluation, calibration, robustness, multilingual tests, safety evaluations, and domain-specific tests complement it.

Benchmark results are sensitive to prompt format, few-shot examples, scoring method, contamination, and sampling settings. A trustworthy report gives model version, data cutoff where known, evaluation harness, sample size, confidence intervals when possible, and failure examples.

A disciplined experiment

Write a hypothesis before changing one variable; keep a fixed validation set; use multiple seeds for small experiments; log configuration and model artifacts; inspect examples where metrics disagree; and document limitations rather than reporting only the best number.

Suggested learning path and exercises

- Train a small character-level language model and plot train vs validation loss.
- Compare BPE token counts for English, code, and a multilingual sample using one tokenizer.
- Explain why next-token prediction can learn useful representations without directly optimizing truthfulness.
- Create a model card and dataset card for a toy language-model experiment.
- Read BERT, GPT-2, T5, and scaling-law papers; make a table of objectives and intended use.

References and further reading

The references below are starting points. Prefer original papers, official documentation, and careful technical reports. Some papers are historical foundations; others describe techniques whose current implementations may have evolved.

Goodfellow, Bengio & Courville (2016), Deep Learning. https://www.deeplearningbook.org/

LeCun, Bengio & Hinton (2015), Deep learning. Nature. https://doi.org/10.1038/nature14539

Kingma & Ba (2014), Adam: A Method for Stochastic Optimization. https://arxiv.org/abs/1412.6980

Devlin et al. (2018), BERT. https://arxiv.org/abs/1810.04805

Radford et al. (2019), Language Models are Unsupervised Multitask Learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Raffel et al. (2020), T5. https://jmlr.org/papers/v21/20-074.html

Brown et al. (2020), Language Models are Few-Shot Learners. https://arxiv.org/abs/2005.14165

Kaplan et al. (2020), Scaling Laws for Neural Language Models. https://arxiv.org/abs/2001.08361

Hoffmann et al. (2022), Training Compute-Optimal Large Language Models. https://arxiv.org/abs/2203.15556